

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD PERUANA LOS ANDES



FACULTAD INGENIERIA
ESPECIALIDAD: SISTEMAS Y COMPUTACION

Desarrollo Ejercicios _Semana 10

CURSO:

Base de Datos II

DOCENTE:

-Fernández Bejarano Raúl Enrique

ESTUDIANTE:

- Rosales Crispín Aristóteles Onassis

Ciclo:

5°SEMESTRE

HUANCAYO- JUNIN
2026

PROYECTO 1: Creación y distribución de archivos físicos

Enunciado:

Cree la base de datos denominada (Ejercicios_10_R) distribuyendo sus archivos en rutas físicas independientes del servidor de la siguiente manera: un archivo principal (primario), un archivo complementario (secundario) con el fin de optimizar el rendimiento en consultas de registros históricos, y el archivo de transacciones de manera aislada.

Compruebe la correcta asignación del almacenamiento consultando los metadatos de los archivos físicos generados en el sistema.

Ejercicios propuestos:

1. **Incorporación de un grupo de archivos exclusivo para Índices:** Modifica la estructura de la base de datos (Ejercicios_10_R) agregando un nuevo grupo de archivos llamado IdxGroup con un archivo físico secundario (.ndf). Esto servirá para almacenar de forma aislada todos los índices no agrupados de la base de datos (como IDX_Persona_DNI o DX Tramite Matricula), optimizando la velocidad de las búsquedas

```
SQLQuery1.sql - DE...N0H\Pavilion (76))* -> X
CREATE DATABASE Ejercicios_10_R;
GO

-- Paso 1: Agregar el nuevo filegroup
ALTER DATABASE EjerciciosSem10
ADD FILEGROUP IdxGroup;
GO

-- Paso 2: Agregar archivo físico al filegroup
ALTER DATABASE EjerciciosSem10
ADD FILE (
    NAME          = EjerciciosSem10_IdxDData,
    FILENAME       = 'D:\BaseDatos2026\BasePrincipal_idx.ndf',
    SIZE           = 10MB,
    MAXSIZE        = 5GB,
    FILEGROWTH     = 5MB
)
TO FILEGROUP IdxGroup;
GO

SELECT
    fg.name          AS NombreFilegroup,
    df.name           AS NombreArchivoLogico,
    df.physical_name  AS RutaFisica,
    (df.size * 8) / 1024.0 AS TamanoMB
FROM sys.filegroups fg
JOIN sys.database_files df
    ON fg.data_space_id = df.data_space_id
WHERE fg.name = 'IdxGroup';
GO

ALTER INDEX IX_Persona_DNI
ON Academico.Persona
REBUILD WITH (ONLINE = OFF, MOVE TO IdxGroup);
GOUSE Ejercicios_10;
GO
```

JUSTIFICACIÓN TÉCNICA

Al separar los índices no agrupados en un filegroup propio (IdxGroup), el motor de SQL Server puede paralelizar las operaciones de lectura de índices y datos en discos físicamente distintos. Índices como IX_Persona_DNI o IX_Tramite_Matricula son consultados constantemente en búsquedas por DNI y por estado de trámite, por lo que su aislamiento reduce la competencia de cabezas de lectura

BUENAS PRÁCTICAS

- Nombrar el filegroup con un nombre semántico (IdxGroup) y no genérico (FG2).
- Definir MAXSIZE explícito para evitar que el archivo crezca sin control hasta agotar el disco.
- Después de mover índices, ejecutar DBCC SHOWCONTIG o consultar sys.dm_db_index_physical_stats para confirmar que no quedaron fragmentados.

2.-Reubicación de un Filegroup y consulta de espacio disponible: Desarrolla una consulta transaccional avanzada que muestre el espacio total asignado, el espacio utilizado y el espacio libre (en megabytes) para cada uno de los grupos de archivos físicos que componen actualmente la base de datos (Ejercicios_10_R).

```
3
4 SELECT
5     fg.name                                AS GrupoArchivos,
6     df.name                                AS ArchivoLogico,
7     df.physical_name                       AS RutaFisica,
8     df.type_desc                           AS TipoArchivo,
9     CAST((df.size * 8.0) / 1024 AS DECIMAL(10,2)) AS EspacioTotalMB,
10    CAST(
11        (df.size * 8.0 / 1024)
12        - (df.size * 8.0 / 1024)
13        * (FILEPROPERTY(df.name, 'SpaceUsed') * 1.0 / df.size)
14        AS DECIMAL(10,2)
15    ) AS EspacioLibreMB,
16    CAST(
17        FILEPROPERTY(df.name, 'SpaceUsed') * 8.0 / 1024
18        AS DECIMAL(10,2)
19    ) AS EspacioUsadoMB,
20    CAST(
21        100.0 * FILEPROPERTY(df.name, 'SpaceUsed') / NULLIF(df.size, 0)
22        AS DECIMAL(5,2)
23    ) AS PorcentajeUso
24 FROM sys.database_files df
25 LEFT JOIN sys.filegroups fg
26     ON df.data_space_id = fg.data_space_id
27 ORDER BY fg.name, df.name;
28 GO
```

JUSTIFICACIÓN TÉCNICA

La función FILEPROPERTY permite obtener en tiempo real el espacio efectivamente usado dentro de cada archivo, dato que sys.database_files por sí solo no provee directamente (solo expone el tamaño asignado). Combinar ambas fuentes da una vista completa del estado de almacenamiento, lo que es clave para planificar crecimientos y detectar archivos sobredimensionados o que están al límite.

BUENAS PRÁCTICAS

- Ejecutar esta consulta como parte de un job de monitoreo semanal en el SQL Server Agent.
- Alertar cuando PorcentajeUso supere el 80% para anticipar crecimientos automáticos inesperados.

PROYECTO 2:Ajuste de configuración y validación de propiedades

Enunciado:

Consulta las propiedades de colación actuales de la base de datos. Modifica la colación de la base de datos para asegurar el soporte nativo de caracteres en español (tildes y la letra 'Ñ'). Adicionalmente, ajusta el crecimiento automático del archivo primario de datos de manera fija a 20 MB.

Ejercicios propuestos:

1. **Elevación del Nivel de Compatibilidad (Compatibility Level):** Verifica el nivel de compatibilidad actual de (Ejercicios_10_R) y elévalo a la última versión disponible del motor de base de datos para habilitar optimizaciones de consulta avanzadas, como el procesamiento de consultas inteligentes (Intelligent Query Processing).

```
SQLQuery1.sql - DE...N0H\Pavilion (76))*  X
-- 1. Verificar nivel actual
SELECT
    name AS BaseDatos,
    compatibility_level AS NivelActual
FROM sys.databases
WHERE name = 'Ejercicios_10_R';
GO

-- 2. Elevar al nivel 160 (SQL Server 2022 / Azure SQL)
-- Habilita: Intelligent Query Processing, Parameter Sensitive Plan, etc.
ALTER DATABASE Ejercicios_10_R
SET COMPATIBILITY_LEVEL = 160;
GO

-- 3. Verificar el cambio
SELECT
    name AS BaseDatos,
    compatibility_level AS NivelNuevo
FROM sys.databases
WHERE name = 'Ejercicios_10_R';
GO
```

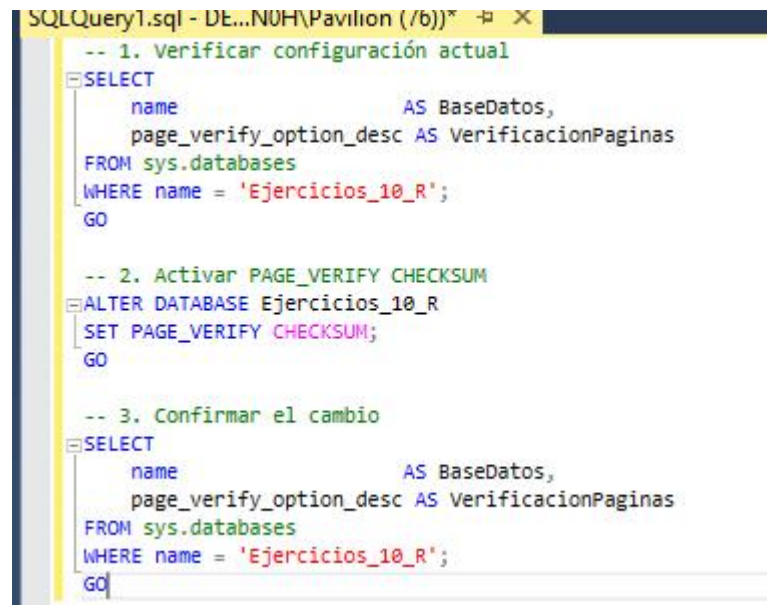
JUSTIFICACIÓN TÉCNICA

El nivel 160 activa el Intelligent Query Processing (IQP) que incluye características como Adaptive Memory Grant Feedback, Batch Mode on Rowstore y Parameter Sensitive Plan optimization. Para Semana_10_R, consultas frecuentes sobre Tramite.Tramite con múltiples filtros (estado, tipo, coordinación) se benefician del plan adaptativo, especialmente durante periodos masivos de inicio de ciclo académico.

BUENAS PRÁCTICAS

- Antes de elevar el nivel, ejecutar el Query Store para capturar planes base y detectar regresiones post-cambio.
- Elevar por etapas si la BD tiene muchos procedimientos almacenados (140 → 150 → 160).
- Usar ALTER DATABASE SCOPED CONFIGURATION para habilitar/deshabilitar funcionalidades IQP de forma granular si alguna consulta regresa.

2.-Activación de la propiedad PAGE VERIFY CHECKSUM para protección contra corrupción: Configura la propiedad de verificación de páginas de la base de datos (Ejercicios_10_R) en modo CHECKSUM para asegurar que el motor detecte de inmediato cualquier corrupción de datos a nivel físico causada por fallas en el disco o en el sistema operativo de almacenamiento.



```
SQLQuery1.sql - DE...N0H\Pavilion (/b)) * X
-- 1. Verificar configuración actual
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPaginas
FROM sys.databases
WHERE name = 'Ejercicios_10_R';
GO

-- 2. Activar PAGE_VERIFY CHECKSUM
ALTER DATABASE Ejercicios_10_R
SET PAGE_VERIFY CHECKSUM;
GO

-- 3. Confirmar el cambio
SELECT
    name AS BaseDatos,
    page_verify_option_desc AS VerificacionPaginas
FROM sys.databases
WHERE name = 'Ejercicios_10_R';
GO
```

JUSTIFICACIÓN TÉCNICA

CHECKSUM calcula una firma criptográfica de cada página (8 KB) al momento de escribirla en disco. Cuando SQL Server lee esa página posteriormente, recalcula el checksum y lo compara. Si no coincide, lanza el error 824 ("I/O error"), deteniendo la operación antes de que datos corruptos lleguen a la aplicación. Esto es crítico para (Ejercicios_10_R) dado que almacena diplomas y resoluciones con valor legal.

BUENAS PRÁCTICAS

- Habilitar alertas sobre el error 824 en SQL Server Agent para notificación inmediata.
- Combinar con DBCC CHECKDB semanal para auditoría completa de integridad.
- La alternativa TORN_PAGE_DETECTION es menos costosa pero solo detecta escrituras parciales, no corrupción silenciosa de disco.

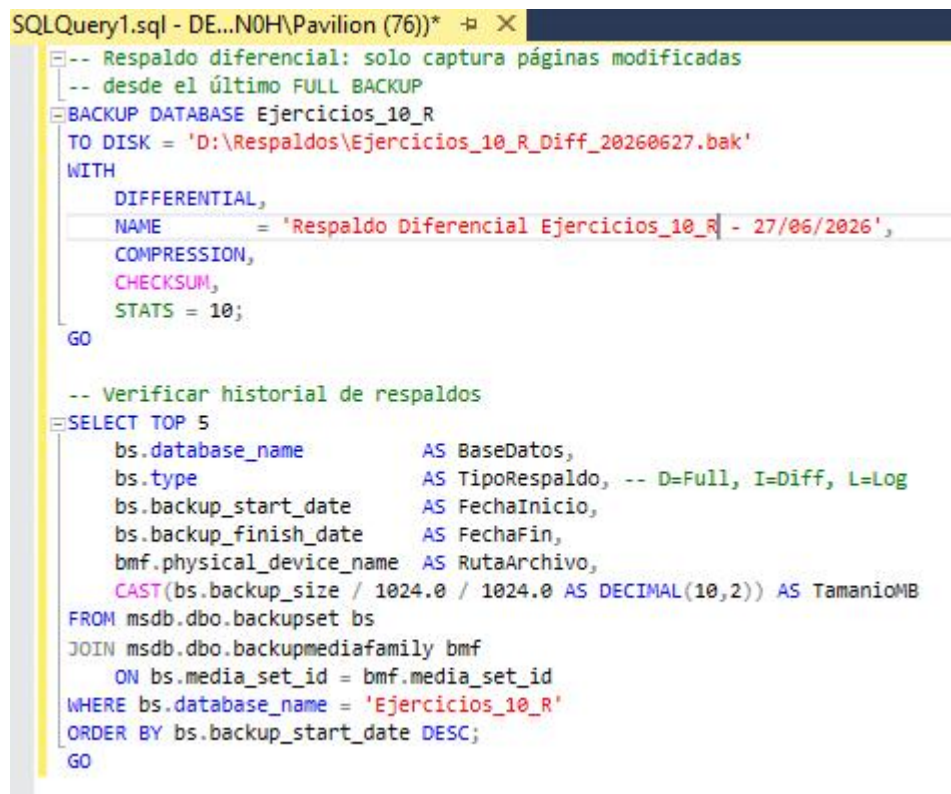
PROYECTO 3: Definición de modelo de recuperación y respaldo

Enunciado:

Cambia de forma secuencial el modelo de recuperación de la base de datos (Ejercicios_10_R) a SIMPLE y posteriormente a BULK_LOGGED. Explica el impacto de ambos entornos. Finalmente, cambia el modelo a FULL y ejecuta un respaldo completo (BACKUP DATABASE) hacia un directorio seguro de almacenamiento.

• Ejercicios propuestos:

1. **Respaldo diferencial de seguridad para optimización de tiempos:** Con el objetivo de optimizar el espacio en el almacenamiento y acelerar la ventana de mantenimiento diario, ejecuta un respaldo de tipo diferencial (DIFFERENTIAL) de la base de datos (Ejercicios_10_R) , guardándolo en un archivo específico.



```
SQLQuery1.sql - DE...N0H\Pavilion (76))* + X
-- Respaldo diferencial: solo captura páginas modificadas
-- desde el último FULL BACKUP
BACKUP DATABASE Ejercicios_10_R
TO DISK = 'D:\Respaldos\Ejercicios_10_R_Diff_20260627.bak'
WITH
    DIFFERENTIAL,
    NAME = 'Respaldo Diferencial Ejercicios_10_R - 27/06/2026',
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Verificar historial de respaldos
SELECT TOP 5
    bs.database_name          AS BaseDatos,
    bs.type                   AS TipoRespaldo, -- D=Full, I=Diff, L=Log
    bs.backup_start_date      AS FechaInicio,
    bs.backup_finish_date     AS FechaFin,
    bmf.physical_device_name  AS RutaArchivo,
    CAST(bs.backup_size / 1024.0 / 1024.0 AS DECIMAL(10,2)) AS TamanoMB
FROM msdb.dbo.backupset bs
JOIN msdb.dbo.backupmediafamily bmf
    ON bs.media_set_id = bmf.media_set_id
WHERE bs.database_name = 'Ejercicios_10_R'
ORDER BY bs.backup_start_date DESC;
GO
```

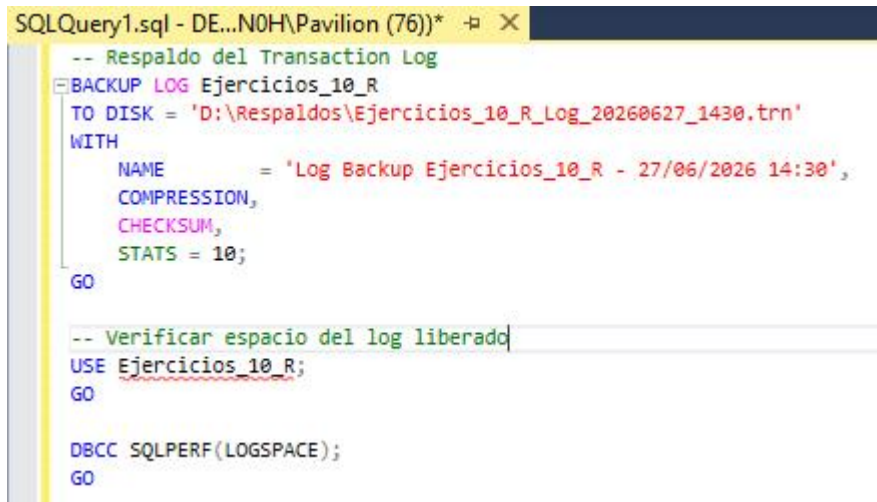
JUSTIFICACIÓN TÉCNICA

Un respaldo diferencial captura únicamente las páginas de datos que cambiaron desde el último backup completo, no desde el diferencial anterior. Esto hace que crezca progresivamente durante la semana pero que la restauración sea siempre de solo dos pasos: Full + último Diff. Para (Ejercicios_10_R es ideal ejecutarlo al cierre de cada día laboral universitario.

BUENAS PRÁCTICAS

- Combinar con un esquema 1 Full (domingo) + 6 Diferenciales (lunes a sábado) + Logs cada hora.
- Guardar los respaldos en una ruta de red distinta al servidor de BD para protegerse ante fallo del disco local.
- Validar el respaldo con RESTORE VERIFYONLY antes de darlo por bueno.

2.- Respaldo periódico del Log de Transacciones (Transaction Log Backup): Dado que la base de datos opera bajo el modelo de recuperación FULL, realiza un respaldo exclusivo del archivo de registro de transacciones (LOG) de (Ejercicios_10_R) para liberar el espacio reutilizable dentro del archivo Ldf y asegurar la posibilidad de recuperar los datos hasta un minuto exacto antes de un fallo.



```
SQLQuery1.sql - DE...N0H\Pavilion (76))* -p X
-- Respaldo del Transaction Log
BACKUP LOG Ejercicios_10_R
TO DISK = 'D:\Respaldos\Ejercicios_10_R_Log_20260627_1430.trn'
WITH
    NAME          = 'Log Backup Ejercicios_10_R - 27/06/2026 14:30',
    COMPRESSION,
    CHECKSUM,
    STATS = 10;
GO

-- Verificar espacio del log liberado
USE Ejercicios_10_R;
GO

DBCC SQLPERF(LOGSPACE);
GO
```

JUSTIFICACIÓN TÉCNICA

En modelo FULL, el log de transacciones (.ldf) solo se puede truncar (liberar el espacio interno para reutilización) cuando se ejecuta un BACKUP LOG. Sin este respaldo, el log crece indefinidamente hasta agotar el disco. El respaldo de log también es el mecanismo que permite la recuperación punto en el tiempo: restaurando el Full + todos los logs hasta el momento exacto del fallo.

BUENAS PRÁCTICAS

- Programar respaldos de log cada 15-60 minutos en producción para minimizar pérdida de datos (RPO).
- Nunca borrar manualmente archivos .trn sin verificar que la cadena de restauración sigue intacta.
- Monitorear log_reuse_wait_desc en sys.databases; si dice LOG_BACKUP, significa que el archivo crece porque no se están haciendo backups de log.

PROYECTO 4: Implementación de roles y usuarios para seguridad

Enunciado:

Crea un esquema de seguridad basado en roles integrados del sistema. Configura el login de servidor y usuario de base de datos VendedorQhatu (renombrado bajo el ecosistema institucional como Operador Tramite), asignándolo al rol db_datawriter para la gestión diaria del flujo administrativo. Asimismo, configura el usuario Consulta Cliente (renombrado como AuditorSunedu) vinculándolo exclusivamente al rol db_datareader.

Ejercicios propuestos:

1. **Creación de un Rol de Base de Datos personalizado (Custom Database Role):** El equipo de secretaría académica necesita un perfil de seguridad intermedio. Crea un rol personalizado dentro de la base de datos llamado RolSecretaria Academica. Concédeles permisos de lectura (SELECT) en el esquema Academico y en el esquema Tramite, pero otórgales también permisos para insertar registros (INSERT) únicamente en la tabla Tramite. Tramite.

```
SQLQuery1.sql - DE...N0H\Pavilion (76))*  + X
-- 1. Crear el rol personalizado
CREATE ROLE RolSecretariaAcademica;
GO

-- 2. Permisos de lectura en los esquemas completos
GRANT SELECT ON SCHEMA::Academico TO RolSecretariaAcademica;
GO
GRANT SELECT ON SCHEMA::Tramite TO RolSecretariaAcademica;
GO

-- 3. Permiso de inserción SOLO en Tramite.Tramite
GRANT INSERT ON Tramite.Tramite TO RolSecretariaAcademica;
GO

-- 4. Crear usuario de ejemplo y asignarle el rol
CREATE LOGIN SecretariaUser01 WITH PASSWORD = 'Secr3GDFGt$2026!';
GO
CREATE USER SecretariaUser01 FOR LOGIN SecretariaUser01;
GO
ALTER ROLE RolSecretariaAcademica ADD MEMBER SecretariaUser01;
GO

-- 5. Verificar permisos del rol
SELECT
    dp.class_desc          AS TipoObjeto,
    OBJECT_NAME(dp.major_id) AS Objeto,
    dp.permission_name      AS Permiso,
    dp.state_desc           AS Estado
FROM sys.database_permissions dp
JOIN sys.database_principals pr
    ON dp.grantee_principal_id = pr.principal_id
WHERE pr.name = 'RolSecretariaAcademica';
GO
```

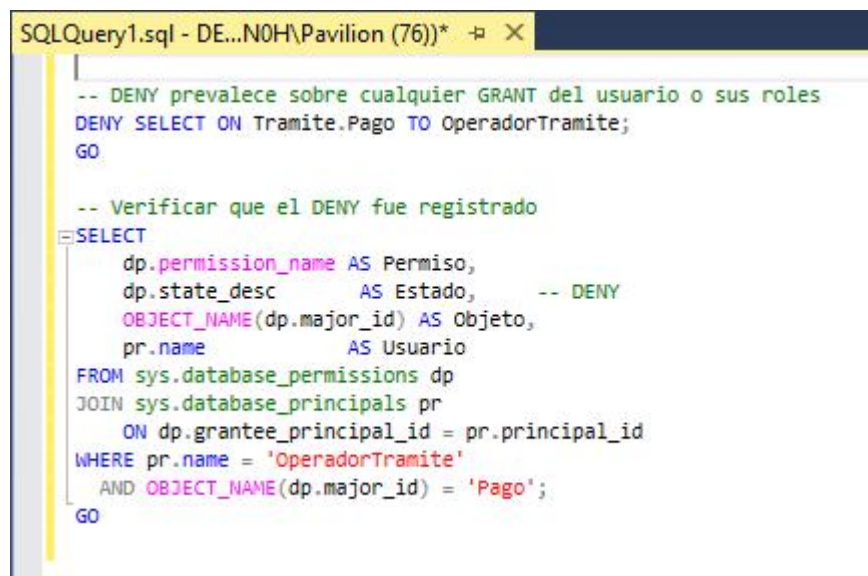

JUSTIFICACIÓN TÉCNICA

Un rol personalizado agrupa permisos reutilizables, lo que permite agregar nuevos miembros del personal de secretaría sin tener que reasignar permisos individualmente. El acceso SELECT a nivel de esquema completo (en lugar de tabla por tabla) facilita el mantenimiento cuando se añaden nuevas tablas a los esquemas Academico o Tramite. El INSERT granular solo sobre Tramite. Tramite respeta el principio de mínimo privilegio.

BUENAS PRÁCTICAS

- Preferir permisos a nivel de rol sobre permisos directos a usuarios; si el usuario cambia de puesto, basta removerlo del rol.
- Revocar explícitamente los permisos al eliminar un usuario antes de hacer DROP USER.
- Auditar periódicamente los miembros de cada rol con sys.database_role_members.

2.-Denegación explícita de acceso mediante DENY: Para proteger la información financiera de la universidad, aplica una restricción estricta de seguridad bloqueando explícitamente al usuario Operador Tramite cualquier lectura o visualización de datos de la tabla Tramite. Pago, prevaleciendo sobre cualquier otro rol que tenga asignado.



```
SQLQuery1.sql - DE...N0H\Pavilion (76)) *  X
-- DENY prevalece sobre cualquier GRANT del usuario o sus roles
DENY SELECT ON Tramite.Pago TO OperadorTramite;
GO

-- Verificar que el DENY fue registrado
SELECT
    dp.permission_name AS Permiso,
    dp.state_desc      AS Estado,      -- DENY
    OBJECT_NAME(dp.major_id) AS Objeto,
    pr.name            AS Usuario
FROM sys.database_permissions dp
JOIN sys.database_principals pr
    ON dp.grantee_principal_id = pr.principal_id
WHERE pr.name = 'OperadorTramite'
AND OBJECT_NAME(dp.major_id) = 'Pago';
GO
```

JUSTIFICACIÓN TÉCNICA

DENY es la instrucción de mayor jerarquía en el modelo de seguridad de SQL Server: anula cualquier GRANT concedido directa o indirectamente (por rol). Para proteger la tabla Tramite.Pago que contiene montos y recibos de pagos universitarios, el DENY SELECT garantiza que aunque OperadorTramite sea añadido a un rol que tenga SELECT general, nunca podrá ver esa información financiera.

BUENAS PRÁCTICAS

- Documentar todos los DENY explícitos en un registro de seguridad, ya que son difíciles de detectar cuando se diagnostican problemas de acceso.
- Combinar con Row-Level Security (RLS) si se requiere control a nivel de fila en lugar de tabla completa.
- Revisar periódicamente que los DENY siguen siendo necesarios cuando cambia la estructura organizacional.

PROYECTO 5: Configuración granular de permisos en el módulo académico

Enunciado:

Aplica el principio de mínimo privilegio de forma granular. Concede al usuario GerenteQhatu (actualizado a CoordinadorFacultad) acceso exclusivo de solo lectura (SELECT) a la vista analítica Evaluacion.V_Nota Final Sustentacion. En paralelo, revoca explícitamente al rol o usuario CajeroQhatu (actualizado a Asistente Tramite) el permiso de modificación (UPDATE) sobre la tabla crítica Tramite.Pago.

Ejercicios propuestos:

1. **Permisos granulares a nivel de columnas (Column-Level Permissions):** El equipo de desarrollo técnico necesita realizar estadísticas de contacto con los docentes, pero por normativas de privacidad no deben ver información sensible. Concede al usuario Practicante Sistemas acceso de lectura (SELECT) únicamente a las columnas Id Persona, Nombres, Apellido Paterno, y Correo de la tabla Academico.Persona.

```
SQLQuery1.sql - DE...N0H\Pavilion (76))* -> X
USE master;
GO
CREATE LOGIN PracticanteSist WITH PASSWORD = 'Pract$2026!';
GO

USE Ejercicios_10_R;
GO
CREATE USER PracticanteSistemas FOR LOGIN PracticanteSist;
GO

-- Permisos granulares a nivel de columna
GRANT SELECT ON Academico.Persona (IdPersona, Nombres, ApellidoPaterno, Correo)
TO PracticanteSistemas;
GO

-- Verificar permisos de columna
SELECT
    col.name AS Columna,
    dp.permission_name AS Permiso,
    dp.state_desc AS Estado,
    pr.name AS Usuario
FROM sys.database_permissions dp
JOIN sys.database_principals pr
    ON dp.grantee_principal_id = pr.principal_id
JOIN sys.columns col
    ON dp.major_id = col.object_id
    AND dp.minor_id = col.column_id
WHERE pr.name = 'PracticanteSistemas'
    AND OBJECT_NAME(dp.major_id) = 'Persona';
GO
```

JUSTIFICACIÓN TÉCNICA

Los permisos a nivel de columna permiten compartir datos de contacto (necesarios para estadísticas) sin exponer información sensible como DNI, Teléfono o tipo de contrato del docente. Es el mecanismo más granular de control de acceso a datos en SQL Server y cumple con normativas de protección de datos personales (LOPD/GDPR equivalentes en Perú).

BUENAS PRÁCTICAS

- Documentar exactamente qué columnas son consideradas sensibles por política de privacidad institucional.
- Considerar como alternativa una vista con solo las columnas permitidas; es más mantenible cuando el conjunto de columnas autorizadas cambia.
- Nunca otorgar permisos de columna sobre campos como DNI, contraseñas o datos biométricos a perfiles operativos o de práctica.

2.-Concesión del permiso EXECUTE sobre Procedimientos Almacenados: Concede al usuario Operador Tramite permisos exclusivos para ejecutar el procedimiento almacenado Tramite.SP_Registrar Tramite, garantizando que pueda procesar nuevas solicitudes sin necesidad de darle accesos directos de escritura sobre las tablas involucradas.

```
SELECT TOP 5
    qs.total_worker_time / 1000 AS CPU_Total_ms,
    qs.execution_count AS Ejecuciones,
    qs.total_worker_time
    / NULLIF(qs.execution_count, 0) AS CPU_Promedio_us,
    qs.total_elapsed_time
    / NULLIF(qs.execution_count, 0) AS TiempoTotal_Promedio_us,
    qs.total_logical_reads
    / NULLIF(qs.execution_count, 0) AS LecturasLogicas_Prom,
    SUBSTRING(qt.text, 1, 500) AS TextoConsulta,
    qp.query_plan AS PlanEjecucion
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE qt.dbid = DB_ID('Ejercicios_10_R')
ORDER BY qs.total_worker_time DESC;
GO
```

JUSTIFICACIÓN TÉCNICA

Este patrón se llama "ejecución por cadena de propiedad" (Ownership Chaining): el SP pertenece al mismo esquema que las tablas que usa, por lo que SQL Server no verifica los permisos de OperadorTramite sobre las tablas base durante la ejecución del SP. El usuario puede ejecutar la lógica de negocio completa (incluyendo validaciones y transacciones) sin que se le concedan permisos directos de escritura sobre Tramite.Tramite o Academico.Matricula.

BUENAS PRÁCTICAS

- Usar este patrón como estándar para todos los usuarios operativos: acceden a los datos solo a través de SPs, nunca directamente a las tablas.
- Combinar con EXECUTE AS dentro del SP para mayor control sobre el contexto de seguridad en tiempo de ejecución.
- Auditar las ejecuciones de SPs críticos con Extended Events o triggers DDL

PROYECTO 6: Identificación y solución de procesos lentos y bloqueos

Enunciado:

Durante periodos masivos de carga académica del flujo de egresados, el servidor experimenta lentitud. Describe los indicadores clave de rendimiento a monitorizar mediante la interfaz gráfica del Activity Monitor. Simultáneamente, desarrolla una consulta avanzada en T-SQL utilizando vistas de administración dinámica (DMVs) para rastrear bloqueos de recursos en tiempo real en la base de datos.

Ejercicios propuestos:

1. **Identificación de consultas que consumen mayor tiempo de CPU:** Escribe una consulta en T-SQL que analice los planes de ejecución que se encuentran almacenados en la memoria caché del servidor. El script debe listar las 5 consultas que han consumido la mayor cantidad de tiempo de procesador (CPU) en la base de datos Ejercicios_10_R

```
SELECT TOP 5
    qs.total_worker_time / 1000 AS CPU_Total_ms,
    qs.execution_count AS Ejecuciones,
    qs.total_worker_time
    / NULLIF(qs.execution_count, 0) AS CPU_Promedio_us,
    qs.total_elapsed_time
    / NULLIF(qs.execution_count, 0) AS TiempoTotal_Promedio_us,
    qs.total_logical_reads
    / NULLIF(qs.execution_count, 0) AS LecturasLogicas_Prom,
    SUBSTRING(qt.text, 1, 500) AS TextoConsulta,
    qp.query_plan AS PlanEjecucion
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) qt
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle) qp
WHERE qt.dbid = DB_ID('Ejercicios_10_R')
ORDER BY qs.total_worker_time DESC;
GO
```

JUSTIFICACIÓN TÉCNICA

sys.dm_exec_query_stats almacena estadísticas acumuladas de todos los planes de ejecución que siguen en caché. Ordenar por total_worker_time (tiempo de CPU acumulado) identifica las consultas que han consumido más recursos desde el último reinicio o limpieza del caché. Para (Ejercicios_10_R), las consultas más costosas probablemente involucran JOINS entre Tramite.Tramite, Academico.Matricula y Academico.Persona sin filtros de índice adecuados.

BUENAS PRÁCTICAS

- Analizar el query_plan XML en el Query Plan Viewer de SSMS para identificar table scans o index scans costosos.
- Complementar con Query Store (sys.query_store_plan) que persiste estadísticas incluso después de reinicios.
- Una consulta con alto CPU_Promedio_us pero pocas ejecuciones es candidata a optimización de plan; una con alto CPU_Total_ms pero muchas ejecuciones necesita primero reducir la frecuencia de llamadas

2.- Rastreo del uso de los Índices en las consultas (Index Usage Stats): Implementa una consulta analítica en T-SQL para verificar el uso real de los índices no agrupados creados en los esquemas de Ejercicios_10_R, identificando si hay índices que no están siendo utilizados en las búsquedas pero que siguen consumiendo espacio en disco.

```
SQLQuery1.sql - DE...N0H\Pavilion (76))* - X
SELECT
    OBJECT_SCHEMA_NAME(i.object_id) AS Esquema,
    OBJECT_NAME(i.object_id) AS Tabla,
    i.name AS NombreIndice,
    i.type_desc AS TipoIndice,
    ISNULL(us.user_seeks, 0) AS Seeks,
    ISNULL(us.user_scans, 0) AS Scans,
    ISNULL(us.user_lookups, 0) AS Lookups,
    ISNULL(us.user_updates, 0) AS Actualizaciones,
    ISNULL(us.user_seeks, 0)
    + ISNULL(us.user_scans, 0)
    + ISNULL(us.user_lookups, 0) AS Totallecturas,
    us.last_user_seek AS UltimoSeek,
    us.last_user_scan AS UltimoScan,
    CASE
        WHEN ISNULL(us.user_seeks, 0)
            + ISNULL(us.user_scans, 0)
            + ISNULL(us.user_lookups, 0) = 0
        THEN 'INDICE NO UTILIZADO - CANDIDATO A ELIMINAR'
        WHEN ISNULL(us.user_updates, 0) > (
            ISNULL(us.user_seeks, 0)
            + ISNULL(us.user_scans, 0)
            + ISNULL(us.user_lookups, 0)) * 5
        THEN 'Alto COSTO MANTENIMIENTO vs BENEFICIO'
        ELSE 'EN USO'
    END AS Diagnostico
FROM sys.indexes i
LEFT JOIN sys.dm_db_index_usage_stats us
    ON us.object_id = i.object_id
    AND us.index_id = i.index_id
    AND us.database_id = DB_ID('Ejercicios_10_R')
WHERE i.type_desc = 'NONCLUSTERED'
    AND OBJECT_SCHEMA_NAME(i.object_id) IN
        ('Academico', 'Tramite', 'Evaluacion', 'Documento', 'Catalogo')
ORDER BY Totallecturas ASC, Actualizaciones DESC;
GO
```

JUSTIFICACIÓN TÉCNICA

sys.dm_db_index_usage_stats se resetea con cada reinicio del servicio SQL Server, por lo que sus datos son válidos desde el último inicio del servidor. Un índice con cero seeks/scans/lookups pero muchas actualizaciones es perjudicial: consume espacio en disco, degrada el rendimiento de INSERT/UPDATE/DELETE, pero no está siendo aprovechado por ninguna consulta. Para Semana_10_R, índices como IX_Eval_Etapa sobre EvaluacionTrabajo podrían no estar siendo usados si las consultas siempre filtran por IdTrabajo primero.

BUENAS PRÁCTICAS

- No eliminar un índice basándose solo en estas estadísticas si el servidor fue reiniciado recientemente; esperar al menos 4-6 semanas de operación normal.
- Los índices con Diagnostico = 'ALTO COSTO MANTENIMIENTO vs BENEFICIO' son candidatos a ser rediseñados o eliminados.
- Combinar con sys.dm_db_index_physical_stats para detectar adicionalmente cuáles índices en uso están también fragmentados.

PROYECTO 7: Automatización de respaldos y limpieza periódica con SQL Server Agent

Enunciado:

Diseña los scripts transaccionales necesarios para automatizar tareas rutinarias a través de SQL Server Agent. El primer proceso ejecutará un respaldo completo programado diariamente de la base de datos. El segundo proceso se encargará de purgar semanalmente todos aquellos registros de auditoría almacenados en la tabla de observaciones que superen los 15 días de antigüedad y cuyo estado ya se encuentre resuelto.

- Ejercicios propuestos:

1. **Notificación de alertas ante fallos en los Jobs del Agente:** Escribe el script transaccional para configurar un operador del sistema llamado DBA Alertas en el Agente de SQL Server, de tal manera que el servidor envíe correos electrónicos automáticos si el Job de respaldo diario de la universidad falla.

```
-- 1. Crear el operador DBA_Alertas
EXEC sp_add_operator
    @name          = N'DBA_Alertas',
    @enabled       = 1,
    @email_address  = N'dba.alertas@universidad.edu.pe',
    @weekday_pager_start_time = 80000,
    @weekday_pager_end_time   = 200000;
GO

-- 2. Agregar notificación de fallo al job existente
EXEC sp_add_notification
    @alert_name      = N'', -- se usa con el job directamente
    @operator_name    = N'DBA_Alertas',
    @notification_method = 1; -- 1=email, 2=pager, 4=net send
GO

-- Alternativa: notificación directa en el job
EXEC sp_update_job
    @job_name          = N'Semana10_Respaldo_Completo_Diario',
    @notify_level_email = 2, -- notificar solo en FALLO
    @notify_email_operator_name = N'DBA_Alertas';
GO

-- 3. Crear alerta para el error 3041 (fallo de backup)
EXEC sp_add_alert
    @name          = N'Alerta_Fallo_Backup_Semana_10',
    @message_id     = 3041,
    @severity       = 0,
    @enabled       = 1,
    @delay_between_responses = 300, -- 5 min entre alertas repetidas
    @notification_message = N'ALERTA: El respaldo de Semana_10 ha fallado. Verificar inmediatamente.',
    @job_name       = N'';
GO

EXEC sp_add_notification
    @alert_name      = N'Alerta_Fallo_Backup_Semana_10',
    @operator_name    = N'DBA_Alertas',
    @notification_method = 1;
GO
```

JUSTIFICACIÓN TÉCNICA

El error 3041 es el código SQL Server para "fallo general de backup". Combinando la alerta sobre ese error específico con el operador DBA_Aleras configurado con correo, el DBA recibe una notificación inmediata sin necesidad de revisar manualmente los logs del Agent. Para Seman_10_R, donde los respaldos son críticos por el valor legal de los diplomas, este mecanismo garantiza que ningún fallo pase desapercibido.

BUENAS PRÁCTICAS

- Configurar el perfil de Database Mail (sp_configure 'Database Mail XPs') antes de crear operadores de correo.
- Definir @delay_between_responses para evitar inundación de correos si el error se repite en bucle.
- Crear un segundo operador como respaldo (DBA_Aleras_Backup) para redundancia si el correo principal no está disponible.

2.- Automatización de la Reconstrucción de Índices Fragmentados: Desarrolla un script de mantenimiento automatizado para evaluar los índices de la base de datos Seman_10_R y ejecutar una reconstrucción completa (ALTER INDEX REBUILD) en aquellos índices que superen un 30% de fragmentación física, garantizando la velocidad de las consultas.

```
-- Crear el SP de mantenimiento de indices
CREATE OR ALTER PROCEDURE dbo.SP_RebuildFragmentedIndexes
    @Umbra1Fragmentacion DECIMAL(5,2) = 30.0
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Esquema      NVARCHAR(128);
    DECLARE @Tabla        NVARCHAR(128);
    DECLARE @Indice        NVARCHAR(128);
    DECLARE @Fragmentacion DECIMAL(5,2);
    DECLARE @SQL           NVARCHAR(500);

    -- Cursor sobre indices fragmentados
    DECLARE cur_indices CURSOR FOR
    SELECT
        OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
        OBJECT_NAME(ps.object_id)        AS Tabla,
        i.name                            AS Indice,
        ps.avg_fragmentation_in_percent  AS Fragmentacion
    FROM sys.dm_db_index_physical_stats(
        DB_ID('Seman_10_R'), NULL, NULL, NULL, 'LIMITED'
    ) ps
    JOIN sys.indexes i
        ON ps.object_id = i.object_id
        AND ps.index_id = i.index_id
    WHERE ps.avg_fragmentation_in_percent > @Umbra1Fragmentacion
        AND ps.index_id > 0                -- excluir heaps
        AND ps.page_count > 100           -- evitar indices muy pequeños
    ORDER BY ps.avg_fragmentation_in_percent DESC;

    OPEN cur_indices;
    FETCH NEXT FROM cur_indices INTO @Esquema, @Tabla, @Indice, @Fragmentacion;

    WHILE @@FETCH_STATUS = 0
```

JUSTIFICACIÓN TÉCNICA

sys.dm_db_index_physical_stats reporta el porcentaje de fragmentación de cada índice. La fragmentación física degrada el rendimiento porque el motor debe leer más páginas de disco (no contiguas) para recorrer el índice. Un umbral del 30% es el estándar de la industria: entre 5-30% se recomienda REORGANIZE (menos costoso, online), sobre 30% se recomienda REBUILD (más completo pero bloquea la tabla en ONLINE=OFF).

BUENAS PRÁCTICAS

- En producción con SQL Server Enterprise, usar ONLINE = ON para no bloquear la tabla durante la reconstrucción.
- Filtrar por page_count > 100 para evitar reconstruir índices sobre tablas pequeñas donde la fragmentación no tiene impacto real.
- Programar este job en horario de mínima carga (madrugada del domingo) ya que REBUILD es costoso en I/O y CPU.

PROYECTO 8: Reestructuración física y alteración de esquemas

Enunciado:

Con el fin de expandir el control analítico del proceso de sustentaciones y registros, adapta el ejercicio agregando de manera segura una columna denominada PrioridadAtencion de tipo entero (INT) a la estructura de la tabla Tramite. Tramite. Posteriormente, y emulando la corrección de requerimientos del sistema, elimina por completo la columna redundante Observaciones de dicha tabla.

- **Ejercicios propuestos:**

1. **Incorporación de Restricciones CHECK complejas post-creación:** Modifica la tabla Tramite Pago agregando una nueva restricción de comprobación (CHECK CONSTRAINT) para asegurar que ningún registro acepte combinaciones donde el campo Monto sea menor o igual a cero, obligando a que todo pago ingresado al sistema sea siempre una cantidad positiva.

```
SELECT
    cc.name          AS NombreConstraint,
    cc.definition AS Definicion
FROM sys.check_constraints cc
WHERE OBJECT_NAME(cc.parent_object_id) = 'Pago'
AND OBJECT_SCHEMA_NAME(cc.parent_object_id) = 'Tramite';
GO

-- Si no existe, crear la restricción
ALTER TABLE Tramite.Pago
ADD CONSTRAINT CHK_Pago_Monto_Positivo
CHECK (Monto > 0);
GO

-- WITH CHECK valida también los datos ya existentes en la tabla
ALTER TABLE Tramite.Pago
WITH CHECK CHECK CONSTRAINT CHK_Pago_Monto_Positivo;
GO

-- Verificar
SELECT
    cc.name          AS Constraint,
    cc.definition AS Definicion,
    cc.is_disabled AS Deshabilitada
FROM sys.check_constraints cc
WHERE OBJECT_NAME(cc.parent_object_id) = 'Pago';
GO
```

JUSTIFICACIÓN TÉCNICA

WITH CHECK CHECK CONSTRAINT hace que SQL Server valide la restricción contra todos los registros existentes en la tabla antes de habilitarla definitivamente. Si se usa solo ADD CONSTRAINT sin WITH CHECK, la restricción queda en estado is_not_trusted = 1, lo que impide que el optimizador la use para mejorar planes de consulta. Para Semana_10_R es crítico que ningún pago tenga monto ≤ 0 , lo que representaría un error contable.

BUENAS PRÁCTICAS

- Nombrar las restricciones con la convención CHK_[Tabla]_[Descripcion] para identificarlas fácilmente.
- Deshabilitar temporalmente la restricción (NOCHECK) solo durante migraciones masivas de datos históricos, rehabilitándola inmediatamente después.
- Incluir restricciones CHECK en el script de creación original para que apliquen desde el primer registro.

2.-Modificación del tipo de datos de una columna con validación de nulidad: Por requerimientos de ampliación de cobertura de texto, modifica la columna NroRecibo de la tabla Tramite.Pago, elevando su tipo de datos de VARCHAR(30) a VARCHAR(60) y asegurando que mantenga su propiedad de permitir valores nulos (NULL).

```
-- 1. Verificar estado actual de la columna
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite'
    AND TABLE_NAME = 'Pago'
    AND COLUMN_NAME = 'NroRecibo';
GO

-- 2. Modificar el tipo (VARCHAR de 30 a 60, manteniendo NULL)
ALTER TABLE Tramite.Pago
    ALTER COLUMN NroRecibo VARCHAR(60) NULL;
GO

-- 3. Verificar el cambio
SELECT
    COLUMN_NAME,
    DATA_TYPE,
    CHARACTER_MAXIMUM_LENGTH,
    IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Tramite'
    AND TABLE_NAME = 'Pago'
    AND COLUMN_NAME = 'NroRecibo';
GO
```

JUSTIFICACIÓN TÉCNICA

Ampliar el tamaño de un VARCHAR en SQL Server es una operación de metadatos para columnas NULL (no reescribe las filas físicas), lo que la hace rápida y de mínimo impacto. Si la columna fuera NOT NULL, sí requeriría una reescritura parcial. La ampliación de NroRecibo de 30 a 60 caracteres responde al requerimiento de que algunos recibos del sistema financiero universitario tienen formatos más largos (incluyendo prefijos de año, sede y consecutivo).

BUENAS PRÁCTICAS

- Nunca reducir el tamaño de una columna en producción sin auditar primero que los datos existentes cabrán en el nuevo tamaño (consulta `MAX(LEN(NroRecibo))`).
- Ampliar siempre en potencias de 2 o valores redondos (30→60, no 30→37) para facilitar revisiones futuras.
- Documentar el cambio en el diccionario de datos del proyecto Semana_10_R.

PROYECTO 9: Implementación de auditoría forense automatizada mediante Triggers

Enunciado:

Es necesario auditar por políticas de protección de datos personales toda eliminación física sobre el padrón de personas. Diseña e implementa una tabla de historial denominada Academico Auditoria Persona. Acto seguido, desarrolla un desencadenador (Trigger) de nivel de fila para el evento DELETE en la tabla Academico.Persona, el cual capturará los datos eliminados, el usuario del sistema que ejecutó la acción y el momento exacto del incidente.

Ejercicios propuestos:

1. **Trigger de Auditoría para el control de Modificaciones (UPDATE):** Desarrolla un trigger llamado TR_Auditoria_ModificarPago en la tabla Tramite.Pago. Este trigger debe activarse tras cualquier modificación de registros (AFTER UPDATE), guardando en una tabla de auditoría el valor anterior del monto cobrado, el nuevo valor ingresado y el usuario responsable del cambio.

```
-- 1. Tabla de auditoria de pagos
CREATE TABLE Tramite.AuditoriaPago (
    IdAuditoriaPago INT NOT NULL IDENTITY(1,1),
    IdPago INT NOT NULL,
    IdTramite INT NOT NULL,
    MontoAnterior DECIMAL(10,2) NOT NULL,
    MontoNuevo DECIMAL(10,2) NOT NULL,
    TipoPagoAnterior VARCHAR(25) NULL,
    TipoPagoNuevo VARCHAR(25) NULL,
    UsuarioCambio VARCHAR(100) NOT NULL,
    FechaCambio DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    HostName VARCHAR(100) NULL,
    CONSTRAINT PK_AuditoriaPago PRIMARY KEY (IdAuditoriaPago)
);
GO

-- 2. Trigger AFTER UPDATE
CREATE OR ALTER TRIGGER Tramite.TR_Auditoria_ModificarPago
ON Tramite.Pago
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- Solo registrar si el Monto o TipoPago realmente cambió
    IF UPDATE(Monto) OR UPDATE(TipoPago)
    BEGIN
        INSERT INTO Tramite.AuditoriaPago (
            IdPago, IdTramite,
            MontoAnterior, MontoNuevo,
            TipoPagoAnterior, TipoPagoNuevo,
            UsuarioCambio, HostName
        )
        SELECT
            i.IdPago,
            i.IdTramite,
            d.Monto, -- valor ANTES del update (tabla deleted)
```

JUSTIFICACIÓN TÉCNICA

En triggers AFTER UPDATE, SQL Server provee tanto inserted (valores nuevos) como deleted (valores anteriores). La función UPDATE(columna) permite que el trigger actúe solo cuando columnas específicas (Monto, TipoPago) son modificadas, evitando registros innecesarios si se actualiza otro campo irrelevante. Para Semana_10_R, cualquier modificación de montos de pago debe quedar registrada para auditorías contables y prevención de fraude interno.

BUENAS PRÁCTICAS

- Usar IF UPDATE(columna) para filtrar solo los cambios relevantes y reducir el volumen de datos de auditoría.
- Nunca modificar directamente la tabla inserted o deleted dentro del trigger; son de solo lectura.
- Implementar una política de retención para la tabla AuditoriaPago (ej: purgar registros mayores a 5 años) para controlar su crecimiento.

2. **-Trigger de control de seguridad institucional (Restricción de Horarios):** Implementa un trigger de seguridad de tipo INSTEAD OF en la tabla Academico.Docente para bloquear cualquier intento de eliminación o modificación de registros los fines de semana (sábados y domingos), cancelando la transacción automáticamente por motivos de seguridad informática.

```
SQLQuery1.sql - DE...N0H\Pavilion (76))* -> X
CREATE OR ALTER TRIGGER Academico.TR_Docente_RestriccionFinesDeSemana
ON Academico.Docente
INSTEAD OF DELETE, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    -- DATEPART(dw,...) depende del @@DATEFIRST configurado.
    -- Con SET DATEFIRST 7 (por defecto en SQL Server): 1=domingo, 7=sábado
    DECLARE @DiaSemana TINYINT = DATEPART(dw, GETDATE());

    -- Bloquear si es sábado (7) o domingo (1)
    IF @DiaSemana IN (1, 7)
    BEGIN
        RAISERROR(
            'SEGURIDAD INSTITUCIONAL: No se permiten modificaciones o eliminaciones de registros los fines de semana.',
            16, 1
        );
        RETURN; -- no se ejecuta ninguna acción; INSTEAD OF no hace nada si no se escribe en la tabla
    END;

    -- Si no es fin de semana, ejecutar la operación original
    -- Para DELETE:
    IF EXISTS (SELECT 1 FROM deleted) AND NOT EXISTS (SELECT 1 FROM inserted)
    BEGIN
        DELETE d
        FROM Academico.Docente d
        JOIN deleted del ON d.IdDocente = del.IdDocente;
    END;

    -- Para UPDATE:
    IF EXISTS (SELECT 1 FROM inserted) AND EXISTS (SELECT 1 FROM deleted)
    BEGIN
        UPDATE d
        SET
            d.GradoAcademico = i.GradoAcademico,
            d.CodigoORCID = i.CodigoORCID,

```

JUSTIFICACIÓN TÉCNICA

Un trigger INSTEAD OF reemplaza completamente la operación que lo dispara. Si el trigger no ejecuta explícitamente el DELETE o UPDATE en su cuerpo, la operación original nunca se realiza, lo que lo convierte en el mecanismo ideal para bloqueo condicional. Para Semana_10_R, esto protege el padrón de docentes de modificaciones accidentales o maliciosas durante fines de semana, cuando el personal autorizado de RR.HH. no está disponible para supervisar.

BUENAS PRÁCTICAS

- Documentar explícitamente en comentarios que el trigger INSTEAD OF debe reimplementar la operación para días hábiles.
- Considerar que @@DATEFIRST puede variar según la configuración del servidor; usar SET DATEFIRST 7 al inicio del trigger para garantizar consistencia.
- Para mayor robustez, complementar con una restricción por horario también a nivel de login usando sys.login_time en una política de seguridad.

PROYECTO 10: Simulación de recuperación y restauración completa ante contingencias

Enunciado:

Simula un escenario catastrófico provocado por error humano mediante el borrado accidental de datos críticos en el catálogo institucional. Posteriormente, ejecuta la restauración completa de la base de datos Semana_10_R utilizando el respaldo de seguridad (.bak) generado previamente en el Proyecto

3. Finaliza validando la correcta restitución y consistencia de los datos mediante una consulta de verificación.

Ejercicios propuestos:

1. Restauración parcial de tablas mediante exportación e inserción controlada:

Ante un error operativo, se borraron accidentalmente los datos de la tabla de parámetros Catalogo Estado Tramite. En lugar de restaurar toda la base de datos y perder las transacciones del día, utiliza comandos avanzados de consulta entre bases de datos (Cross-Database Queries) para recuperar la información desde una copia de respaldo montada en el mismo servidor bajo el nombre Semana_10_R Auxiliar.

```
USE Ejercicios_10_R
GO

-- Insertar solo los registros que faltan (evitar duplicados)
INSERT INTO Sema_10_R.Catalogo.EstadoTramite (
    IdEstadoTramite, Descripcion, EsTerminal
)
SELECT
    aux.IdEstadoTramite,
    aux.Descripcion,
    aux.EsTerminal
FROM GradosTitulos_Auxiliar.Catalogo.EstadoTramite aux
WHERE NOT EXISTS (
    SELECT 1
    FROM Sema_10_R.Catalogo.EstadoTramite prod
    WHERE prod.IdEstadoTramite = aux.IdEstadoTramite
);
GO

-- Paso 3: Verificar recuperación
SELECT * FROM Catalogo.EstadoTramite ORDER BY IdEstadoTramite;
GO

-- Paso 4: Desmontar BD auxiliar cuando ya no se necesite
USE master;
GO

DROP DATABASE GradosTitulos_Auxiliar;
GO
```

JUSTIFICACIÓN TÉCNICA

Las consultas entre bases de datos (Cross-Database Queries) en SQL Server usan la sintaxis [BaseDatos].[Esquema].[Tabla], lo que permite acceder a datos de una BD auxiliar restaurada sin afectar las transacciones activas de la BD de producción. Esta técnica es ideal cuando el daño es puntual (una tabla vaciada) y la restauración completa de toda la BD implicaría perder las transacciones del día. Para Semana_10_R, recuperar solo Catalogo.EstadoTramite preserva todos los trámites registrados durante el día del incidente.

BUENAS PRÁCTICAS

- La BD auxiliar debe restaurarse con archivos físicos en rutas diferentes a las de la BD de producción para no generar conflictos.
- Eliminar la BD auxiliar tan pronto como se complete la recuperación parcial para liberar espacio en disco.
- Documentar en el plan de contingencia que este procedimiento requiere espacio libre en disco equivalente al doble del tamaño de la BD.

2.-Verificación de la consistencia física con DBCC CHECKDB: Tras completar una restauración de emergencia en el servidor, ejecuta el comando de diagnóstico avanzado DBCC CHECKDB para realizar una auditoría completa de la integridad estructural y lógica de las páginas de datos e índices de Semana_10_R, asegurando que no existan errores ocultos en el almacenamiento.

```
-- 1. DBCC CHECKDB completo con supresión de mensajes informativos
-- Verifica: páginas de datos, índices, integridad referencial, catálogos del sis
DBCC CHECKDB('Semana_10_R')
WITH
    NO_INFOMSGS,      -- solo muestra errores, no mensajes OK
    ALL_ERRORMSG,     -- muestra todos los errores sin truncar
    DATA_PURITY;     -- verifica valores de columnas fuera de rango
GO

-- 2. Verificación rápida solo de tablas (sin índices) para menor impacto
DBCC CHECKTABLE('Catalogo.EstadoTramite') WITH NO_INFOMSGS;
GO
DBCC CHECKTABLE('Tramite.Tramite')          WITH NO_INFOMSGS;
GO
DBCC CHECKTABLE('Academico.Persona')        WITH NO_INFOMSGS;
GO

-- 3. Verificar el catálogo del sistema de MATIAZ
DBCC CHECKCATALOG('MATIAZ') WITH NO_INFOMSGS;
GO

-- 4. Ver el historial de las últimas ejecuciones de CHECKDB
SELECT
    database_id,
    last_clean_dbcc_done      AS UltimaVerificacionLimpia,
    DATEDIFF(DAY, last_clean_dbcc_done, GETDATE()) AS DiasDesdeUltimaVerif
FROM sys.dm_db_persisted_sku_features;
GO

-- Alternativa para ver historial en el log del sistema:
DBCC DBINFO('Semana_10_R') WITH TABLERESULTS, NO_INFOMSGS;
GO
```

JUSTIFICACIÓN TÉCNICA

DBCC CHECKDB es el comando de diagnóstico más completo de SQL Server; verifica la integridad estructural de todas las páginas de datos (comprobando checksums si PAGE_VERIFY CHECKSUM está activo), la coherencia de todos los índices y la validez del catálogo del sistema. DATA_PURITY agrega la verificación de que los valores de columnas están dentro del rango de su tipo de dato, detectando corrupción que a veces pasa el checksum de página. Para Semana_10_R, ejecutarlo post-restauración garantiza que no hay corrupción silenciosa antes de volver a producción.

BUENAS PRÁCTICAS

- Programar DBCC CHECKDB al menos semanalmente en horario de mínima carga; en BDs grandes puede tardar horas.
- Si CHECKDB reporta errores, NO intentar reparación (REPAIR_ALLOW_DATA_LOSS) sin antes hacer un respaldo; la reparación puede implicar pérdida de datos.
- Monitorear el campo last_clean_dbcc_done en msdb para asegurarse de que las verificaciones se están completando correctamente.

